

1140. Stone Game II -

Explanation

Problem Link

Description

Alice and Bob continue their games with piles of stones. There are a number of piles arranged in a row, and each pile has a positive integer number of stones $piles[i]$. The objective of the game is to end with the most stones.

Alice and Bob take turns, with Alice starting first.

On each player's turn, that player can take all the stones in the **first** x remaining piles, where $1 \leq x \leq 2M$. Then, we set $M = \max(M, X)$. Initially, $M = 1$.

The game continues until all the stones have been taken.

Assuming Alice and Bob play optimally, return the **maximum** number of stones Alice can get.

Example 1:

Input: $piles = [3,1,2,5,7]$

Output: 10

Explanation: Alice takes first pile, then Bob takes the second pile, then Alice takes the next two piles and Bob takes the last remaining pile. This makes Alice's score $3 + 2 + 5 = 10$, which is the maximum Alice can get.

Example 2:

Input: piles = [4,3,2,5,10]

Output: 11

Constraints:

- $1 \leq \text{piles.length} \leq 100$
- $1 \leq \text{piles}[i] \leq 10,000$

Topics

Company Tags

Please upgrade to NeetCode Pro to view company tags.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Dynamic Programming (Memoization)** - Caching optimal results for each game state to avoid redundant computation
- **Game Theory / Minimax** - Understanding two-player optimal play where one player maximizes and the other minimizes
- **Suffix Sum** - Precomputing cumulative sums from right to left to efficiently calculate remaining stones
- **Multi-dimensional State Tracking** - Managing DP states with multiple parameters (position, M value, whose turn)

1. Dynamic Programming (Top-Down)

Intuition

This is a two-player game where Alice and Bob take turns picking piles from the front. The key insight is that both players play optimally, meaning Alice tries to maximize her score while Bob tries to minimize Alice's score. We can model this using recursion with memoization, tracking whose turn it is, the current index, and the value of M. When it's Alice's turn, she picks the option that maximizes her total; when it's Bob's turn, he picks the option that minimizes Alice's total.

Algorithm

1. Use a recursive function `dfs(alice, i, M)` where `alice` indicates whose turn it is, `i` is the current pile index, and `M` is the maximum number of piles that can be taken.
2. Base case: if `i == n`, return 0 (no more piles).
3. If it's Alice's turn, try taking `x` piles (1 to `2M`) and maximize the result by adding the stones taken plus the recursive call for Bob's turn.

4. If it's Bob's turn, try taking x piles and minimize the result since Bob wants to minimize Alice's score.
5. Update M to $\max(M, x)$ after each move.
6. Memoize results using a 3D cache with dimensions $[2][n][n+1]$.
7. Return $\text{dfs}(\text{True}, 0, 1)$ to get Alice's maximum score.

Stone Game II - Dynamic Programming with Game Theory

Step 1 / 15

Piles

2

0

7

1

9

2

4

3

4

4

Memoization Cache

Empty

Call Stack

bottom

$\text{dfs}(\text{True}, 0, 1)$

top

Current: $\text{alice}=\text{True}, i=0, M=1$

$\text{res}=0, \text{total}=0$

Initialize with $\text{piles}=[2,7,9,4,4]$, call $\text{dfs}(\text{alice}=\text{True}, i=0, M=1)$

Upgrade to Pro to unlock visualizations

0.5x1x2x

```
public class Solution {
    private int[][][] dp;

    public int stoneGameII(int[] piles) {
        int n = piles.length;
        dp = new int[2][n][n + 1];
        for (int[] layer : dp) {
            for (int[] row : layer) {
                Arrays.fill(row, -1);
            }
        }

        return dfs(1, 0, 1, piles);
    }

    private int dfs(int alice, int i, int M, int[] piles) {
        if (i == piles.length) return 0;
        if (dp[alice][i][M] != -1) return dp[alice][i][M];

        int res = alice == 1 ? 0 : Integer.MAX_VALUE;
        int total = 0;

        for (int X = 1; X <= 2 * M; X++) {
            if (i + X > piles.length) break;
            total += piles[i + X - 1];
            if (alice == 1) {
                res = Math.max(res, total + dfs(0, i + X, Math.max(M, X), piles));
            } else {
                res = Math.min(res, dfs(1, i + X, Math.max(M, X), piles));
            }
        }

        dp[alice][i][M] = res;
        return res;
    }
}
```

Expand

Time & Space Complexity

- Time complexity:
- $O(n^3)$
- $O(n^2)$
- $O(n)$
- $O(1)$
- Space complexity:
- $O(n^2)$
- $O(n)$
- $O(1)$
- $O(1)$

2. Dynamic Programming (Top-Down) + Suffix Sum

Intuition

Instead of tracking both players separately, we can simplify by focusing on a single player's perspective. At each position, the current player wants to maximize their own score. The trick is that whatever stones remain after the current player's turn will be split optimally by the opponent. Using suffix sums, if the current player takes some stones, their score equals those stones plus whatever remains minus the opponent's optimal result from the remaining position.

Algorithm

1. Precompute suffix sums where `suffix_sum[i]` is the total stones from index `i` to the end.
2. Define `dfs(i, M)` to return the maximum stones the current player can get starting at index `i` with parameter `M`.
3. Base case: if `i == n`, return `0`.
4. For each choice `X` from `1` to `2M`, the current player can take `suffix_sum[i] - dfs(i + X, max(M, X))`. This works because the current player gets all remaining stones minus what the opponent will optimally take.
5. Memoize with a 2D cache of size `[n][n+1]`.
6. Return `dfs(0, 1)` to get Alice's maximum score.

Stone Game II - Dynamic Programming with Memoization

Step 1 / 14

piles

2

0

7

1

9

2

4

3

n

4

dp

0

1

2

3	
4	
0	
1	
2	
3	
suffix_sum	
0	
0	
0	
1	
0	
2	
4	
3	

Initialize variables: n=4, create dp matrix 4x5 filled with None, and initialize suffix_sum array

Upgrade to Pro to unlock visualizations

0.5x1x2x

```

public class Solution {
    private int[][] dp;
    private int[] suffixSum;

    public int stoneGameII(int[] piles) {
        int n = piles.length;
        dp = new int[n][n + 1];
        for (int i = 0; i < n; i++) {
            Arrays.fill(dp[i], -1);
        }
    }

```



```

        suffixSum = new int[n];
        suffixSum[n - 1] = piles[n - 1];
        for (int i = n - 2; i >= 0; i--) {
            suffixSum[i] = piles[i] + suffixSum[i + 1];
        }

        return dfs(0, 1);
    }

    private int dfs(int i, int M) {
        if (i == suffixSum.length) return 0;
        if (dp[i][M] != -1) return dp[i][M];

        int res = 0;
        for (int X = 1; X <= 2 * M; X++) {
            if (i + X > suffixSum.length) break;
            res = Math.max(res, suffixSum[i] - dfs(i + X, Math.max(M, X)));
        }

        return dp[i][M] = res;
    }
}

```

Expand

Time & Space Complexity

- Time complexity:
- $O(n^3)$
- $O(n^3)$
- 3
-)
- Space complexity:
- $O(n^2)$
- $O(n^2)$

- 2
-)

3. Dynamic Programming (Bottom-Up)

Intuition

We can convert the top-down approach to bottom-up by filling the DP table from the end of the piles array backward. At each position, we compute the optimal result for both Alice and Bob, considering all possible moves. Working backward ensures that when we compute a state, all states it depends on have already been computed.

Algorithm

1. Create a 3D DP array $dp[2][n+1][n+1]$ where the first dimension represents the player (1 for Alice, 0 for Bob).
2. Iterate from $i = n-1$ down to 0, and for each M from 1 to n .
3. For Alice's turn, initialize $dp[1][i][M] = 0$ and try taking x piles (1 to $2M$), computing $total + dp[0][i+x][\max(M, x)]$ and taking the maximum.
4. For Bob's turn, initialize $dp[0][i][M] = \text{infinity}$ and try taking x piles, computing $dp[1][i+x][\max(M, x)]$ and taking the minimum.
5. Return $dp[1][0][1]$ for Alice's maximum score starting at position 0 with $M=1$.

Stone Game II - Dynamic Programming

Step 1 / 14

Piles

2

0

7

1

9

2

4

3

4

4

DP[player][i][M]

0

1

2

3

4

0

0

0

0

0

0

1

0

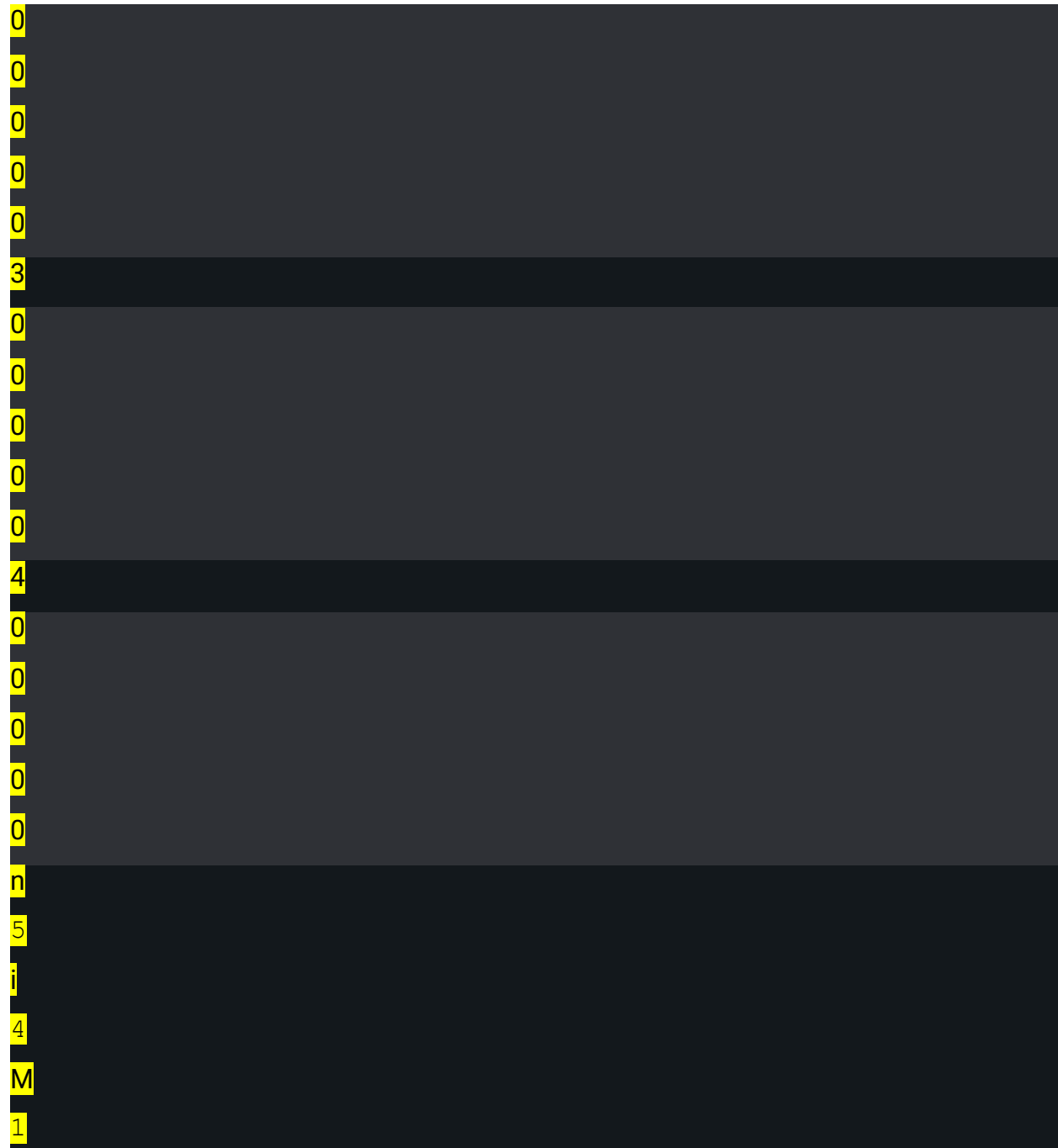
0

0

0

0

2



Initialize 3D DP array with dimensions 2×5×5 for input piles=[2,7,9,4,4].

Upgrade to Pro to unlock visualizations

0.5x1x2x

```
public class Solution {  
    public int stoneGameII(int[] piles) {
```

```

int n = piles.length;
int[][][] dp = new int[2][n + 1][n + 1];

for (int i = n - 1; i >= 0; i--) {
    for (int M = 1; M <= n; M++) {
        int total = 0;
        dp[1][i][M] = 0;
        dp[0][i][M] = Integer.MAX_VALUE;

        for (int X = 1; X <= 2 * M; X++) {
            if (i + X > n) break;
            total += piles[i + X - 1];
            dp[1][i][M] = Math.max(dp[1][i][M], total + dp[0][i + X][Math.max(M, X)]);
            dp[0][i][M] = Math.min(dp[0][i][M], dp[1][i + X][Math.max(M, X)]);
        }
    }
}

return dp[1][0][1];
}
}
}

```

Expand

Time & Space Complexity

- Time complexity:
- $O(n^3)$
- $O(n^3)$
- 3
-)
- Space complexity:
- $O(n^2)$
- $O(n^2)$
- 2

-)

4. Dynamic Programming (Bottom-Up) + Suffix Sum

Intuition

Combining the bottom-up approach with the suffix sum optimization gives us a cleaner solution. Since we treat both players symmetrically (each maximizes their own score), we only need a 2D DP table. The suffix sum lets us compute how many stones the current player gets by subtracting the opponent's optimal result from the remaining total.

Algorithm

1. Precompute suffix sums where $\text{suffix_sum}[i]$ is the total stones from index i to the end.
2. Create a 2D DP array $\text{dp}[n+1][n+1]$ initialized to 0.
3. Iterate from $i = n-1$ down to 0, and for each M from 1 to n .
4. For each X from 1 to $2M$ (while $i + X \leq n$), compute $\text{dp}[i][M] = \max(\text{dp}[i][M], \text{suffix_sum}[i] - \text{dp}[i+X][\max(M, X)])$.
5. Return $\text{dp}[0][1]$ as Alice's maximum score.

Stone Game II - Dynamic Programming

Step 1 / 18

piles

2

0

7	
1	
9	
2	
4	
3	
4	
4	
n	
5	
suffix_sum	
0	
0	
0	
1	
0	
2	
0	
3	
0	
4	
dp	

Initialize with piles = [2, 7, 9, 4, 4] and n = 5.

Upgrade to Pro to unlock visualizations

0.5x1x2x

```
public class Solution {
    public int stoneGameII(int[] piles) {
        int n = piles.length;
```

```

int[] suffixSum = new int[n];
suffixSum[n - 1] = piles[n - 1];
for (int i = n - 2; i >= 0; i--) {
    suffixSum[i] = piles[i] + suffixSum[i + 1];
}

int[][] dp = new int[n + 1][n + 1];

for (int i = n - 1; i >= 0; i--) {
    for (int M = 1; M <= n; M++) {
        for (int X = 1; X <= 2 * M; X++) {
            if (i + X > n) break;
            dp[i][M] = Math.max(dp[i][M], suffixSum[i] - dp[i + X][Math.max(M, X)]);
        }
    }
}

return dp[0][1];
}

```

Expand

Time & Space Complexity

- Time complexity:

- $O(n^3)$

- $O(n^3)$

- $O(n^3)$

- $O(n^3)$

- Space complexity:

- $O(n^2)$

- $O(n^2)$

- $O(n^2)$



Common Pitfalls

Misunderstanding the M Parameter Update

A frequent error is updating M incorrectly after a move. The rule is $M = \max(M, X)$ where X is the number of piles taken, not $M = X$ or $M = M + X$. Failing to take the maximum means M might decrease, which violates the game rules and leads to wrong answers.

Confusing Whose Score to Track

Since both players play optimally but only Alice's score matters for the answer, it is easy to mix up the logic. When using the minimax approach, Alice maximizes her score while Bob minimizes it. When using the suffix sum trick, both players maximize their own score, and the recursion naturally handles the alternation.

Off-by-One Errors in Loop Bounds

The player can take X piles where $1 \leq X \leq 2 \cdot M$. A common mistake is iterating X from 0 to $2 \cdot M$ or from 1 to $2 \cdot M - 1$. Ensure your loop correctly covers exactly the valid range of moves, and check that $i + X$ does not exceed the array length before accessing elements.

